

ПРИНЦИПЫ ПОСТРОЕНИЯ ВЫСОКОНАГРУЖЕННЫХ СИСТЕМ

Мониторинг высоконагруженных систем

Семинар 1 · Кирилл Дикалин

Собственная VM → нагрузочный тест → проверяемый диагноз

Что именно мы собираемся доказать?

Наблюдение

Какие запросы, за какое время и где измерены?

Объяснение

Какой механизм совместим с данными?

Проверка

Какой опыт отличит его от другой причины?

Процесс жив и CPU невысок. Пользователь всё равно может ждать.

До команды запишите прогноз. После — результат и ограничение вывода.

Два файла конфигурации, разные задачи

На ноутбуке	На вашей VM
<code>cloud/env.example</code> → <code>cloud/env.local</code>	<code>lab/env.example</code> → <code>lab/.env</code>
Профиль, облако, каталог, зона	Секреты приложения и Grafana
Имя стенда, SSH <code>/32</code> , пути ключей	Число слотов и размер очереди
Читают наши cloud-скрипты	Читает Docker Compose

Шаблон показывает структуру. Локальный файл содержит ваши значения.

Токен `ус` остаётся в профиле ноутбука. На VM его не передаём.

Проверяем цель до создания ресурсов

```
source cloud/env.local
yc --profile "$YC_PROFILE" resource-manager folder get \
  --id "$YC_FOLDER_ID"
APPROVE_CLOUD_CREATE=NO bash cloud/provision.sh
```

Без `APPROVE_CLOUD_CREATE=YES` — только проверки и план.

Один человек: **1 VM · 4 vCPU · 8 GB RAM · 30 GB SSD.**

Собственные сеть, подсеть и security group. Наружу открыт только SSH.

Создаём сеть и новую машину

Network



Subnet + zone



Security group



VM + disk

```
yc ... vpc network create
yc ... vpc subnet create
yc ... vpc security-group create
yc ... compute instance create
```

Полные исполняемые команды: `docs/01_from_zero.md`, раздел 3.

Вход: TCP/22 с вашего `/32`. Выход: установка пакетов и загрузка образов.

`cloud/.state/<имя>/` хранит ID именно созданных ресурсов.

Что происходит при первом запуске

cloud-init



Пользователь lab



Docker + Compose

`render_cloud_init.py` вкладывает в user-data **публичный SSH-ключ**

и содержимое `install-docker.sh`.

Установка: подписанный apt-репозиторий Docker; без `curl | sh`.

Ещё не запущены: API, Prometheus, Grafana, k6.

Первая загрузка не мгновенная. Проверяем её результат, а не угадываем готовность по прошедшему времени.

Среднее и p99: разные вопросы

$$980 \times 10 \text{ мс} + 20 \times 1000 \text{ мс}$$

Среднее	p50	p95	p99
29,8 мс	10 мс	10 мс	1000 мс

1 000 наблюдений. В этом примере — nearest-rank.

Перцентиль — граница распределения, не максимум и не средняя задержка медленного процента.

Определяем хороший результат

Корректный HTTP 200 И задержка ≤ 300 мс

Термин	Учебный контракт
SLI	Хорошие / все учитываемые запросы
SLO	Не меньше 99% за 28 дней
SLA	Соглашение об обязательствах и последствиях

Деградация бывает без падения: **медленнее, меньше полезных ответов, ошибки.**

99% / 300 мс / 28 дней — учебное допущение. Минутный график не проверяет весь 28-дневный период.

SSH и туннель: где находится localhost

```
# Ноутбук → ваша ВМ; fingerprint сначала сверяем  
ssh ... lab@"$IP" 'sudo cloud-init status --wait'  
bash cloud/connect.sh
```

Ноутбук

127.0.0.1:3000

Браузер → SSH-туннель

ВМ

127.0.0.1:3000

SSH → опубликованный порт Grafana

Токен облака и приватный SSH-ключ остаются на ноутбуке.

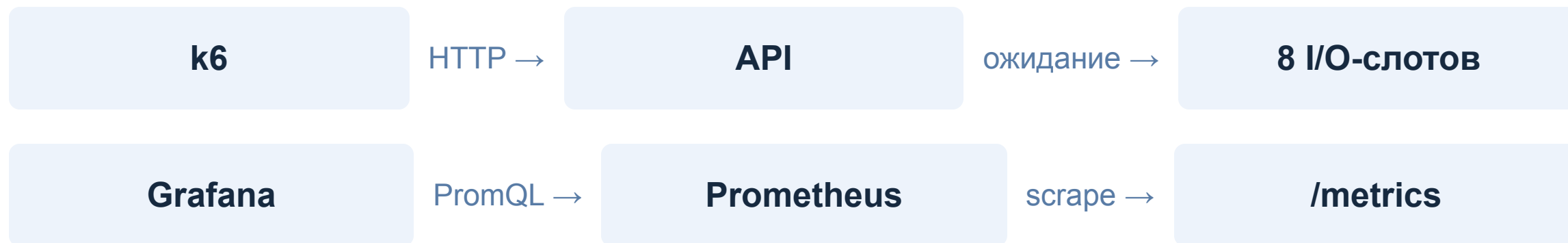
Конфигурация приложения без показа секретов

```
# VM, ~/highload-s1/lab
test -f .env || cp env.example .env
python3 scripts/configure.py
docker compose config --quiet
```

Переменная	Назначение
LAB_ADMIN_TOKEN	Изменение учебного режима
GRAFANA_ADMIN_PASSWORD	Вход в Grafana
WORKERS=8 , QUEUE_LIMIT=32	Параметры ограниченного ресурса

Случайные значения записываются в `.env` с правами `600` , без вывода.

Compose: четыре разных роли



```
docker compose --profile load --profile host pull prometheus grafana k6 node-exporter
docker compose build --pull app
docker compose --profile host up -d app prometheus grafana node-exporter
```

Все компоненты делят ресурсы ВМ. Это модель, не benchmark провайдера.

Готовность проверяем на нескольких уровнях

```
docker compose ps
curl -fsS http://127.0.0.1:8080/healthz
curl -fsS http://127.0.0.1:8080/api/work
bash scripts/check.sh
```

Наблюдение	Что установлено
Контейнер running	Процесс не завершился
/healthz ответил	Ответила отдельная liveness-ручка
Prometheus up=1	Удалось собрать метрики
Успешный /api/work	Успешен этот конкретный запрос

Метрики и стоимость labels

Тип	Пример	Операция
Counter	Завершённые запросы	<code>rate(... [1m])</code> → запросы/с
Gauge	Занятые слоты	Текущее состояние
Histogram	Длительности	Buckets → квантиль или доля

Ограниченные значения: `route`, `status`, `outcome`.

Не добавляем в labels: `user_id`, `request_id`, сырой URL.

Новый набор значений меток — новые ряды.
Для histogram добавляются ряды buckets.

Таймер должен видеть очередь

```
start = time.perf_counter()
try:
    await acquire_slot_with_timeout()
    await perform_work()
finally:
    release_if_acquired()
    duration.observe(time.perf_counter() - start)
```

Раздельно: ожидание слота · удержание слота · общий handler.

При любой развязке: освобождение ресурса и корректные gauges.

Схема; полная обработка статусов и отмен — [lab/app/server.py](#).

Изменение режима — обычный HTTP POST

```
MODE=normal
# Заголовок из .env передаём curl через stdin
printf 'header = "X-Lab-Token: %s"\n' "$TOKEN" | \
  curl --config - --fail-with-body \
  -H 'Content-Type: application/json' \
  -d "{\"mode\":\"$MODE\"}" \
  http://127.0.0.1:8081/admin/mode
```

`mode.sh` добавляет проверки. Не рестартует сервис, не обнуляет counters и не запускает нагрузку.

Первый нагрузочный тест запускаем явно

```
docker compose --profile load run --rm -T \  
  --interactive=false k6 run --quiet \  
  -e RATE=80 -e DURATION=120s -e VUS=160 \  
  -e RUN_LABEL=manual-normal -e EXPECT_NORMAL=1 \  
  /scripts/open.js > "$RUN/summary.json" 2> "$RUN/stderr.log"
```

1 итерация = 1 GET. Планируем 80 поступлений/с, без sleep и retry.

VUS=160 — запас исполнителей. **dropped_iterations** должен быть нулём.

Результат: параметры, время, JSON, stderr и настоящий exit code.

load.sh сохраняет тот же эксперимент

```
# Метка      поступления/с  время  normal-пороги
bash scripts/load.sh baseline 80 120s 1
```

До запуска

Проверка аргументов
и один активный тест

Сам запуск

Та же команда
Docker Compose + k6

После

JSON, stderr, время
и исходный exit code

PromQL: сначала `rate` отдельных counters, затем агрегация.

Согласуем границы измерения

Клиент

Вызов HTTP → ответ
→ проверка тела

`client_wall_ms`

Сервер

Вход в handler → выход
включая ожидание слота

`lab_request_duration_seconds`

Сравниваем: одинаковые исходы, временные окна и единицы.

p99 успешных без доли ошибок — неполная оценка результата.

Редкие задержки меняют хвост

```
bash scripts/mode.sh tail  
bash scripts/load.sh tail 80 120s 0  
bash scripts/collect.sh
```

Каждый 50-й запрос дополнительно удерживает слот **900 мс**.

Сравните: среднее · p95 · p99 · ошибки · хорошие / все.

Почему p95 может почти не измениться,
хотя часть пользователей ждёт значительно дольше?

Квантили нельзя усреднять между машинами

Инстанс	Его запросы	Его p99
A	10 000 × 10 мс	10 мс
B	100 × 1000 мс	1000 мс

Чему равен p99 объединённых 10 100 запросов?

Для агрегации объединяем распределения, а не готовые p99.

Classic histogram: ``rate(bucket)`` → ``sum by (le)``
→ ``histogram_quantile``. Это оценка внутри bucket.

Насыщение ограниченного ресурса

```
bash scripts/mode.sh slow  
bash scripts/load.sh slow 80 120s 0
```

8 слотов · 200 мс работы в слоте · 80 поступлений/с

Очередь ограничена 32 запросами; ожидание слота — до 250 мс.

До графиков: спрогнозируйте успешные ответы/с, отказы и CPU.

Как отличить насыщение пула от ограничения генератора?

От ресурса к пользовательскому результату



Проверяем	Метрика
Ресурс занят?	lab_pool_active , lab_pool_capacity
Запросы ждут?	lab_pool_waiting , wait histogram
Что завершилось?	Все / успешные / хорошие запросы в секунду
Генератор справился?	dropped_iterations , CPU/RAM

Невысокий CPU не исключает насыщения I/O-пула.

Проверяем восстановление при том же потоке

```
bash scripts/mode.sh normal  
bash scripts/load.sh recovery 80 120s 1  
bash scripts/collect.sh
```

Не меняем: rate, длительность, число слотов, программу нагрузки.

Проверяем: очередь уменьшается, отказы исчезают, доля хороших и задержки возвращаются к baseline.

Старые ошибки в минутном окне после переключения не доказывают, что воздействие осталось.

Быстрые ошибки не делают сервис хорошим

```
bash scripts/mode.sh errors
bash scripts/load.sh errors 80 120s 0
bash scripts/collect.sh
bash scripts/mode.sh normal
```

Каждый пятый запрос возвращает HTTP 500 при обычном времени работы.

Одновременно смотрим: up, p99 успешных, 5xx / все, хорошие / все.

Какой из этих графиков отвечает на вопрос пользователя?

Выбор окна — часть эксперимента

k6 summary: вся фаза.

Prometheus `rate(...[1m])`: последняя минута в точке вычисления.

Gauge после теста: уже другое состояние.

```
bash scripts/query.sh good "$EVALUATION_UNIX"
```

Без трафика доля может быть **NaN / No data**, а не 100%.

Исходные файлы и время нужны, чтобы проверить вывод позже.

Самостоятельный тест на своём стенде

```
bash scripts/mode.sh normal  
bash scripts/load.sh own-normal-120 120 120s 1  
bash scripts/collect.sh
```

Меняем один параметр: 80 → 120 планируемых поступлений/с.

До запуска: прогноз и опровергающее наблюдение.

После: поданный поток, latency, good ratio, очередь, вывод.

Не подтвердилось? Различите ограничение приложения и генератора.

Что сохраняет collect.sh

```
RUN=$(cat results/.last-run.txt)
bash scripts/collect.sh "$RUN" # если ещё не собирали этот запуск
```

Источник	Содержимое
k6 + оболочка	Параметры, начало/конец, JSON, stderr, exit code
Prometheus	Выражения в выбранной точке и временной диапазон
API / Docker	Снимки при collect: метрики, targets, логи, ресурсы

Не собираем: `.env`, токен облака, полный `docker inspect`.

Сначала результат, затем удаление

```
# Ноутбук: сначала scp результатов с VM  
# Затем удалить ТОЛЬКО свой стенд по сохранённым ID  
export CONFIRM_DELETE="$LAB_NAME"  
bash cloud/cleanup.sh
```

Порядок: VM → группа безопасности → подсеть → сеть.

Загрузочный диск с `auto-delete` проверяем отдельно.

Остановленная VM и удалённый стенд — разные состояния.
Не удаляем чужие ресурсы по похожему имени.

Результат: воспроизводимый эксперимент

У вас остаются: свой код и конфигурационные шаблоны, отчёт k6, измерения, объяснение и команды повторения.

Проверьте результаты эксперимента:

Условия → прогноз → наблюдения → вывод → ограничение.

Команды экспериментов: `docs/02_experiments.md`.

Что помогло понять механизм?

На каком шаге самостоятельное повторение было затруднено?